

Chapter 6

OOPS Concept: Inheritance

1. What Is Inheritance?

Inheritance is an Object-Oriented Programming concept that allows one class to acquire the properties and methods of another class.

- The class that is inherited from is called the Parent class or Superclass.
- The class that inherits is called the Child class or Subclass.

Purpose: it allows code reusability and method overriding, so existing code can be extended rather than rewritten.

Real-Life Analogy: a child inherits characteristics (like eyes, skin color, talents) from their parents. Similarly, in Java, a subclass inherits variables and methods from a superclass.

2. Advantages of Inheritance

Advantage	Description
Code Reusability	Avoid rewriting common logic.
Easy Maintenance	Updates in the superclass are reflected in subclasses.
Extensibility	Allows adding new features without changing old code.
Method Overriding	Enables polymorphism (same method, different behavior).
Organized Code	Promotes modular and structured programs.

3. Types of Inheritance in Java — Overview

Type	Structure	Description
Single Inheritance	A -> B	One class inherits from another.
Multilevel Inheritance	A -> B -> C	A class inherits from a subclass (inheritance in steps).
Hierarchical Inheritance	A -> B, A -> C	Multiple classes inherit from one superclass.
Multiple Inheritance	A, B -> C	A class inherits from multiple classes (not allowed with classes; allowed with interfaces).
Hybrid Inheritance	Combination	A mix of the above types (not directly supported through classes; achieved using interfaces).

4. Single Inheritance

Definition: when one class inherits from another class using the extends keyword.

```
class Animal {  
    void eat() {
```

```

        System.out.println("Animals can eat.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); // inherited from Animal
        d.bark(); // Dog's own method
    }
}

```

Animals can eat.

Dog barks.

5. Multilevel Inheritance

Definition: a class is derived from another subclass — inheritance happens in steps (grandparent -> parent -> child).

```

class Grandparent {
    void home() {
        System.out.println("Grandparent's house");
    }
}

class Parent extends Grandparent {
    void car() {
        System.out.println("Parent's car");
    }
}

class Child extends Parent {
    void bike() {
        System.out.println("Child's bike");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
    }
}

```

```

        c.home(); // from Grandparent
        c.car();  // from Parent
        c.bike(); // Child's own
    }
}

```

Grandparent's house

Parent's car

Child's bike

6. Hierarchical Inheritance

Definition: when multiple classes inherit from a single parent class.

```

class Animal {
    void eat() {
        System.out.println("All animals eat.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();
        c.eat();
        c.meow();
    }
}

```

All animals eat.

Dog barks.

All animals eat.

Cat meows.

7. Multiple Inheritance (Not Supported with Classes)

Definition: when one class inherits from two or more classes. This causes ambiguity, known as the Diamond Problem — if ClassA and ClassB both define the same method and ClassC inherits from both, Java wouldn't know which version ClassC should use.

Java Restriction: Java does not support multiple inheritance with classes, but allows it using interfaces, since interfaces (traditionally) only declare methods without a conflicting implementation.

Example — Multiple Inheritance via Interfaces

```
interface A {  
    void show();  
}  
interface B {  
    void display();  
}  
  
class C implements A, B {  
    public void show() {  
        System.out.println("Show from A");  
    }  
    public void display() {  
        System.out.println("Display from B");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.show();  
        obj.display();  
    }  
}
```

Show from A

Display from B

8. Hybrid Inheritance

Definition: a combination of two or more types of inheritance (like Hierarchical + Multiple). Java doesn't directly support hybrid inheritance through classes, but allows it via interfaces.

Example 1 — Hybrid via Interface + Two Implementing Classes + Extension

Structure: interface A is implemented by both B and C; class D then extends B, so D inherits A's method through B and adds its own method.

```
interface A {
```

```

        void showA();
    }

    class B implements A {
        public void showA() {
            System.out.println("Inside B");
        }
    }

    class C implements A {
        public void showA() {
            System.out.println("Inside C");
        }
    }

    class D extends B {
        void displayD() {
            System.out.println("Inside D (Hybrid Inheritance using
interfaces)");
        }
    }

    public class Main {
        public static void main(String[] args) {
            D obj = new D();
            obj.showA();
            obj.displayD();
        }
    }

```

Inside B

Inside D (Hybrid Inheritance using interfaces)

Example 2 — Class Extending a Class While Implementing Another Interface

Structure: class C implements interface A; class D then extends C AND implements interface B — combining single inheritance (extends C) with an additional interface (implements B).

```

interface A {
    void showA();
}

interface B {
    void showB();
}

class C implements A {
    public void showA() {
        System.out.println("Method from Interface A");
    }
}

```

```

    }
}

class D extends C implements B {
    public void showB() {
        System.out.println("Method from Interface B");
    }
    void display() {
        System.out.println("Hybrid Inheritance Example");
    }
}

public class HybridInheritanceDemo {
    public static void main(String[] args) {
        D obj = new D();
        obj.showA();
        obj.showB();
        obj.display();
    }
}

```

Method from Interface A

Method from Interface B

Hybrid Inheritance Example

Example 3 — Real-World: SmartPhone (Device + Phone + Camera)

Structure: SmartPhone extends the base class Device, and implements both the Phone and Camera interfaces — combining class inheritance with multiple interface implementation.

```

interface Phone {
    void makeCall();
}
interface Camera {
    void takePhoto();
}

class Device {
    void powerOn() {
        System.out.println("Device Powered On");
    }
}

class SmartPhone extends Device implements Phone, Camera {
    public void makeCall() {
        System.out.println("Calling...");
    }
    public void takePhoto() {

```

```

        System.out.println("Photo Captured");
    }
    void browseInternet() {
        System.out.println("Browsing Internet");
    }
}

public class HybridInheritanceExample {
    public static void main(String[] args) {
        SmartPhone phone = new SmartPhone();
        phone.powerOn();
        phone.makeCall();
        phone.takePhoto();
        phone.browseInternet();
    }
}

```

Device Powered On
Calling...
Photo Captured
Browsing Internet

Example 4 — Real-World: Online Banking App

Structure: BankAccount is the base class, LoanService and PaymentService are interfaces, and OnlineBankingApp extends BankAccount while implementing both interfaces. This is hybrid inheritance because it combines single inheritance (class) with multiple inheritance through interfaces.

```

// Base class
class BankAccount {
    void accountDetails() {
        System.out.println("Account Holder: John");
        System.out.println("Balance: Rs. 50,000");
    }
}

// Interface for loan services
interface LoanService {
    void applyLoan();
}

// Interface for payment services
interface PaymentService {
    void makePayment();
}

// Derived class

```

```

class OnlineBankingApp extends BankAccount
    implements LoanService, PaymentService {

    public void applyLoan() {
        System.out.println("Loan application submitted.");
    }

    public void makePayment() {
        System.out.println("Payment successful.");
    }

    void checkBalance() {
        System.out.println("Current Balance: Rs. 50,000");
    }
}

// Main class
public class HybridInheritanceDemo {
    public static void main(String[] args) {
        OnlineBankingApp user = new OnlineBankingApp();
        user.accountDetails();
        user.checkBalance();
        user.applyLoan();
        user.makePayment();
    }
}

```

Account Holder: John

Balance: Rs. 50,000

Current Balance: Rs. 50,000

Loan application submitted.

Payment successful.

9. Summary Table — All 5 Types

Type	Structure	Supported in Java	Example
Single	A -> B	Yes	One class inherits from another
Multilevel	A -> B -> C	Yes	Stepwise inheritance
Hierarchical	A -> B, A -> C	Yes	One parent, multiple children
Multiple	A, B -> C	Only via interfaces	class C implements A, B
Hybrid	Mix of types	Only via interfaces	Combination of Hierarchical + Multiple

10. Quick Exercises for Students

- Create a program for single inheritance (Teacher -> MathTeacher).
- Write a program for multilevel inheritance (Person -> Employee -> Manager).
- Demonstrate hierarchical inheritance (Vehicle -> Car, Bike).
- Use interfaces to implement multiple inheritance.
- Design a hybrid inheritance model using interfaces and classes.

11. Tips for Understanding Inheritance

1. Always identify the parent and child first, and check that the relationship reads naturally as "[child] IS-A [parent]" (e.g., "a Dog IS-A Animal"). If that sentence feels forced, inheritance is probably the wrong tool for that relationship.
2. Draw the class diagram before writing code, especially for multilevel, hierarchical, and hybrid inheritance. A quick arrow diagram (A -> B -> C, or A -> B / A -> C) makes it immediate which methods each class actually has access to.
3. Remember the one Java restriction that drives everything about multiple/hybrid inheritance: a class can extend only ONE other class, but can implement as many interfaces as needed. Every 'multiple inheritance' or 'hybrid inheritance' example in this chapter is really just working around that single restriction using interfaces.
4. The Diamond Problem is the reason classes can't multi-inherit: if two parent classes both implement the same method differently, the compiler has no rule for which version the child should use. Interfaces avoid this (in the classic form used here) because they only declare method signatures — the implementing class supplies the one and only body.
5. For hybrid inheritance specifically, look for the pattern: one extends (the single class parent) plus one or more implements (the interfaces) on the same class declaration, e.g. class D extends C implements B. That combination is what makes it 'hybrid' rather than purely single or purely multiple.
6. Trace inherited method calls by walking UP the chain. For multilevel inheritance (Grandparent -> Parent -> Child), a Child object can call home() from Grandparent, car() from Parent, and bike() from itself — because each level below inherits everything above it, not just the immediate parent.
7. In hierarchical inheritance, sibling classes (like Dog and Cat, both extending Animal) do NOT share anything with each other directly — they each independently inherit from Animal, but Dog has no access to Cat's bark()-equivalent methods and vice versa.
8. When a real-world example feels complex (like the SmartPhone or OnlineBankingApp), separate out three questions: (1) what's the ONE base class being extended, (2) what interfaces are being implemented, and (3) what NEW methods does the final class add on top of what it inherited. Answering those three always untangles the hybrid structure.
9. Practice reading the output before running the code: for every inherited method call in main(), trace which class's code actually executes. This is the most reliable way to confirm you understand which methods a given object has access to.